Research Plan: COD Prediction in Wastewater Treatment

Ong Jai Sheng

Core Question: How do different temporal feature engineering strategies (Raw Sensor Baseline vs. Kalman Filtering vs. Kalman+ARIMA) and different modelling architectures (Traditional Machine Learning vs. Deep Learning) affect out-of-sample R^2 performance when predicting wastewater `AT_COD`?

I. Introduction

II. Materials and Methods

A. Dataset and Biological Treatment Process

To maintain industrial confidentiality, the dataset used in this study was derived by applying a controlled stochastic perturbation to operational measurements from an active biological wastewater treatment facility. Specifically, independent Gaussian noise with a standard deviation of 0.5% of each variable's observed range was applied to the original sensor readings, producing a statistically representative simulated dataset that preserves the temporal dynamics and inter-variable relationships of the real process while preventing proprietary data disclosure. The resulting dataset comprised 983 chronological daily observations spanning the full operational record of the facility.

The treatment process follows a four-stage pipeline, illustrated in Fig. 1. Raw industrial wastewater first enters the Process Wastewater Tank (PWT), which acts as a primary collection and homogenisation buffer. The wastewater then passes through Wet Air Oxidation (WAO), a thermochemical pre-treatment stage that breaks down refractory organic compounds prior to biological treatment. The partially oxidised effluent subsequently enters the Equalization Tank (ET), which stabilises flow and load variability before the final biological stage. The aeration tank (AT) constitutes the core activated sludge reactor, where aerobic microorganisms mineralise residual organic matter under controlled aeration.

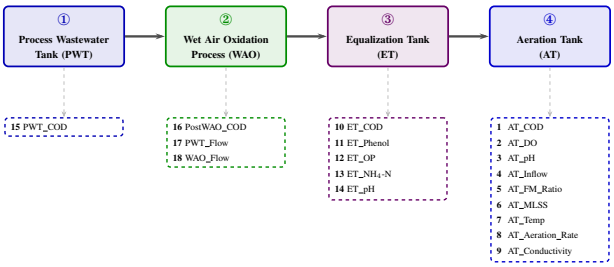

Fig. 1. Four-stage industrial wastewater treatment pipeline. Solid arrows denote the direction of wastewater flow. Dashed arrows indicate the sensor variables collected at each stage.

The sole prediction target of this study is the Chemical Oxygen Demand within the aeration tank (`AT_COD`), which quantifies the residual organic load at the biological treatment stage and is the primary operational indicator of treatment performance. Although Dissolved Oxygen (`AT_DO`) is listed in the raw dataset and is physically relevant to the aeration process, it was excluded from the modelling pipeline during preprocessing and is not a prediction target in this study. Table I summarises the full set of measured variables, their abbreviations, and their roles.

TABLE I
Measured Process Variables Grouped by Treatment Stage

Variable Item		Abbreviation	Description
Aeration Tank Measurements	Aeration Tank COD	AT_COD	Predicted Target
	Aeration Tank Dissolved Oxygen	AT_DO	Predicted Target
	Aeration Tank Inflow Volume	AT_Inflow	Feature
	Aeration Tank Food-to-Microorganism Ratio	AT_FM_Ratio	Feature
	Aeration Tank Mixed Liquor Suspended Solids	AT_MLSS	Feature
	Aeration Tank Temperature	AT_Temp	Feature
	Aeration Tank Aeration Rate	AT_Aeration_Rate	Feature
	Aeration Tank Conductivity	AT_Conductivity	Feature
Equalization Tank Measurements	Equalization Tank COD	ET_COD	Feature
	Equalization Tank Phenol	ET_Phenol	Feature
	Equalization Tank Orthophosphate	ET_OP	Feature
	Equalization Tank Ammonia Nitrogen	ET_NH_4-N	Feature
	Equalization Tank pH	ET_pH	Feature
Process Measurements	Process Wastewater Tank COD Concentration	PWT_COD	Feature
	Post-Wet Oxidation COD Concentration	PostWAO_COD	Feature
	Process Wastewater Tank Direct Flow	PWT_Flow	Feature
	Wet Air Oxidation Process Flow	WAO_Flow	Feature

Prior to modelling, a structured preprocessing pipeline was applied to reduce the 25 available sensor variables to a clean predictor set. First, `AT_DO` was excluded based on domain considerations, and the raw microbial

count columns (c1–c8) were removed due to measurement unreliability, reducing the candidate feature set to 16 variables. Second, Variance Inflation Factor (VIF) analysis was conducted to detect multicollinearity. The variable ET_COD was identified with a VIF score of 21.14, far exceeding the standard threshold of 10, and was subsequently dropped. This yielded a final set of **15 core predictor variables** fed into all downstream models.

A fundamental physical constraint governing this process is the aeration tank's hydraulic retention time of approximately six days. Consequently, influent quality fluctuations required up to six days to manifest in the effluent, meaning contemporaneous sensor readings did not reflect instantaneous cause-and-effect relationships. This temporal delay dictated the time-series architecture of the predictive models: input features were constructed as six-day rolling averages of the process variables to forecast the aeration tank state six days ahead, strictly aligning the machine learning formulation with the physical dynamics of the system.

B. Target-Derived Feature Engineering (The Kalman/ARIMA Pipeline)

Because the 15 core sensor variables retained after preprocessing lacked sufficient direct correlation with the volatile AT_COD target, two autoregressive features were engineered directly from the target time series under a strict walk-forward expanding-window protocol. Each feature was constructed at every time step t using only observations available strictly before t, ensuring no future information contaminated any training sample.

Part 1: Kalman Filter State-Space Formulation. The Kalman-filtered feature AT_CODKSp1 was derived by modelling the AT_COD time series as a two-dimensional linear dynamical system. The state vector $\mathbf{x}_t = [\mu_t,\ \dot{\mu}_t]^\top \in \mathbb{R}^2$ encodes the instantaneous COD level μ_t and its first-order rate of change $\dot{\mu}_t$. The system obeys the discrete-time state-space equations

$$\mathbf{x}_t = \mathbf{F}\,\mathbf{x}_{t-1} + \mathbf{w}_{t-1}, \qquad \mathbf{w}_{t-1} \sim \mathcal{N}(\mathbf{0},\, \mathbf{Q}) \tag{1}$$

$$z_t = \mathbf{H}\,\mathbf{x}_t + v_t, \qquad v_t \sim \mathcal{N}(0,\, R) \tag{2}$$

where z_t is the scalar observed AT_COD at time t; $\mathbf{F} = \left[\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right]$ is the constant-velocity state transition matrix; $\mathbf{H} = [1,\ 0]$ is the observation matrix that extracts the level component; $\mathbf{Q}$ and R are the process and observation noise covariance matrices, respectively; and $\mathbf{w}_{t-1}$, v_t are mutually independent zero-mean Gaussian noise terms. The noise covariances were initialised empirically as the pykalman library defaults: $\mathbf{Q}^{(0)} = 0.1\,\mathbf{I}_2$ and $R^{(0)} = 0.1$, with initial state mean $\hat{\mathbf{x}}_0 = [1,\, 0]^\top$ and initial error covariance $\mathbf{P}_0 = \mathbf{I}_2$. These parameters were then refined at each expanding-window step by the Expectation–Maximisation (EM) algorithm over $n_{\text{iter}} = 10$ iterations.

The filter propagates estimates through a two-step recursive cycle. In the *prediction* step, the prior state estimate and error covariance are projected forward:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\,\hat{\mathbf{x}}_{t-1|t-1} \tag{3}$$

$$\mathbf{P}_{t|t-1} = \mathbf{F}\,\mathbf{P}_{t-1|t-1}\,\mathbf{F}^\top + \mathbf{Q} \tag{4}$$

where $\hat{\mathbf{x}}_{t|t-1}$ is the predicted state estimate at time t given observations up to $t-1$, and $\mathbf{P}_{t|t-1}$ is its associated error covariance matrix. In the *update* step, the new observation z_t is assimilated through the Kalman gain:

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\,\mathbf{H}^\top\big(\mathbf{H}\,\mathbf{P}_{t|t-1}\,\mathbf{H}^\top + R\big)^{-1} \tag{5}$$

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t\big(z_t - \mathbf{H}\,\hat{\mathbf{x}}_{t|t-1}\big) \tag{6}$$

$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\,\mathbf{H})\,\mathbf{P}_{t|t-1} \tag{7}$$

where $\mathbf{K}_t$ is the Kalman gain matrix; $\hat{\mathbf{x}}_{t|t}$ is the posterior state estimate; $\mathbf{P}_{t|t}$ is the updated error covariance; and $\mathbf{I}$ is the 2×2 identity matrix. The first engineered feature, referred to in the dataset as AT_CODKSp1, is denoted mathematically as $f_t^{(1)}$ and defined as the filtered COD level from the previous time step:

$$f_t^{(1)} \;=\; \hat{\mu}_{t-1|t-1} \;=\; \mathbf{e}_1^\top\,\hat{\mathbf{x}}_{t-1|t-1} \tag{8}$$

where $f_t^{(1)}$ is the first Kalman-derived feature at time t, and $\mathbf{e}_1 = [1,\ 0]^\top$ is the first standard basis vector. The

one-step lag arises from the pipeline implementation: at each step t, the Kalman filter is applied to the strictly historical window $\{z_1, \ldots, z_{t-1}\}$, and the last filtered state $\hat{\mathbf{x}}_{t-1|t-1}$ is recorded as the feature for observation t. This ensures $f_t^{(1)}$ contains no same-time or future information.

Part 2: ARIMA Forecasting of the Smoothed Series. The second engineered feature `AT_CODKSp2` was generated by applying an ARIMA model to the Kalman-smoothed series. The smoothed series $\{\tilde{\mu}_\tau\}_{\tau=1}^{t}$, obtained from the Rauch–Tung–Striebel backward smoothing pass on the same expanding window, provides a noise-reduced signal that improves ARIMA stationarity assessment and order selection. An ARIMA(p, d, q) model is defined by

$$\phi(B)\,(1-B)^d\,\tilde{\mu}_t = \theta(B)\,\varepsilon_t \tag{9}$$

where B is the backshift operator such that $B^k\tilde{\mu}_t = \tilde{\mu}_{t-k}$; $(1-B)^d$ denotes d-fold differencing to induce stationarity; $\phi(B) = 1 - \sum_{i=1}^{p}\phi_i\,B^i$ is the autoregressive polynomial of order p; $\theta(B) = 1 + \sum_{j=1}^{q}\theta_j\,B^j$ is the moving-average polynomial of order q; and $\varepsilon_t \sim \mathrm{WN}(0, \sigma^2)$ is a white noise process. The integer orders (p, d, q) were determined automatically at each window step by minimising the Akaike Information Criterion (AIC) via `pmdarima`'s `auto_arima` routine. The resulting one-step-ahead conditional expectation defines the second engineered feature, referred to in the dataset as `AT_CODKSp2` and denoted mathematically as $f_t^{(2)}$:

$$f_t^{(2)} \;=\; \hat{\tilde{\mu}}_{t|t-1} \;=\; \mathbb{E}[\tilde{\mu}_t \mid \mathcal{F}_{t-1}] \tag{10}$$

where $f_t^{(2)}$ is the second ARIMA-derived feature at time t, and $\mathcal{F}_{t-1} = \sigma(\tilde{\mu}_1, \ldots, \tilde{\mu}_{t-1})$ is the filtration generated by the smoothed COD series up to time $t-1$. Consistent with the $f_t^{(1)}$ construction, the ARIMA model at step t is fitted on the smoothed series derived from the strictly historical window $\{z_1, \ldots, z_{t-1}\}$, and `predict(n_periods=1)` returns the one-step forecast of the smoothed COD *at* time t, using only information available before t.

Part 3: Walk-Forward Expanding Window and Analytical Burn-In. Both features were constructed under a walk-forward expanding-window protocol to prevent temporal data leakage. At each time step $t \in \{t_0, t_0{+}1, \ldots, T\}$, the complete pipeline — Kalman EM fitting, filtered and smoothed state estimation, ARIMA order selection, and one-step forecast — was executed exclusively on the strictly historical sub-series $\{z_1, \ldots, z_{t-1}\}$:

$$\big(f_t^{(1)},\; f_t^{(2)}\big) = \mathcal{P}\big(\{z_\tau\}_{\tau=1}^{t-1}\big) \tag{11}$$

where $\mathcal{P}(\cdot)$ denotes the complete feature-generation pipeline. The minimum initialisation period was set to $t_0 = 20$, reflecting the number of observations required for the Kalman EM algorithm to converge to stable noise covariance estimates. Consequently, the first 20 observations were discarded as an analytical burn-in period, yielding a finalised evaluation dataset of 963 strictly out-of-sample chronological observations.

C. Proposed Regression Architecture: Extra Trees

D. Deep Learning Benchmark (Phase 3 LSTM Optimization)

E. Experimental Setup and Evaluation Metrics

III. Results and Discussion

IV. Conclusion

V. References

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In KDD.